# SQL: From Traditional Databases to Big Data

Matin Huseynzade

Computer Engineering Department
Izmir Institute of Technology

# Contents

1. Introduction

2. MapReduce Systems

3. NoSQL Systems

4. NewSQL Systems

5. Conclusion

# Introduction

# SQL: Yesterday's Language in Today's World

**Classroom Reality:**

- SQL on small, static databases
- University/employee examples
- Single-server setups

**Industry Reality:**

- SQL on petabytes of data
- Thousands of distributed nodes
- Real-time streaming data

**If students only learn SQL in traditional systems, are they prepared for today's data world?**

# Paper's Mission

**Challenge and expand how we teach SQL**

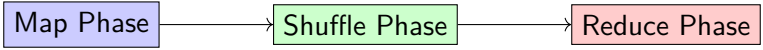
- Present SQL as a **universal data language**
- Cover **Traditional RDBMS**, **MapReduce**, **NoSQL**, and **NewSQL**
- Provide practical teaching resources

# MapReduce Systems

# The Big Data Challenge & MapReduce

**Problem:** Processing massive datasets requires distributed approach

**Solution: MapReduce Framework**




**Analogy:** Counting words in a massive library

- **Map:** Each person counts words in their book
- **Shuffle:** Gather all counts for same word
- **Reduce:** Sum counts for each word

**Hadoop:** Most popular open-source implementation

# The Problem with "Pure" MapReduce

## Complexity Barrier

- Even simple operations require lengthy Java code
- Must specify **how** to do tasks, not just **what** you want
- High barrier for data analysts who know SQL

## Example: Simple Join Operation

- Traditional SQL: `SELECT * FROM A JOIN B ON A.id = B.id`
- MapReduce: 50+ lines of Java code
- Complex coordination between map and reduce phases

# Solution: SQL on Top of MapReduce

**Idea:** Add translation layer to convert SQL into MapReduce jobs

## Apache Hive

- Uses **HiveQL** (SQL-like language)
- Users declare *what* they want
- Hive figures out *how* via complex MapReduce programs
- Transparent conversion: SQL $\rightarrow$ MapReduce

## Teaching Challenge

How to simulate distributed cluster in classroom?

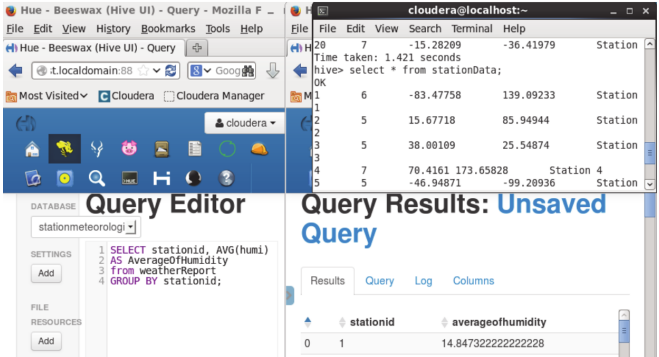
# Teaching Solution: Virtual Machines & Data Generators

## Virtual Machines

- Pre-configured environments
- Cloudera VM, Hortonworks Sandbox
- Simulate multi-node clusters on single machine

## Data Generators

- Create large, customizable datasets
- MStation2: Synthetic weather data
- Adjustable size and distribution



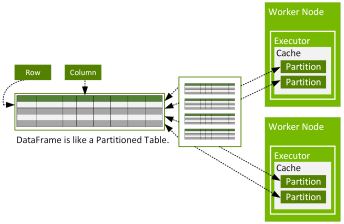
# A Faster Alternative: Apache Spark

**Key Innovation: In-Memory Processing**

- Up to **100x faster** than Hadoop MapReduce
- Avoids disk I/O between operations
- Wider set of processing primitives

**Spark SQL**

- Module for structured data processing
- Uses **DataFrames** (relational tables)
- Created from RDDs, Hive tables, JSON files



# NoSQL Systems

# SQL Convergence in NoSQL Systems

**NoSQL Core Properties:**

- Distributed, fault-tolerant architecture
- Easy to add new nodes (horizontal scaling)
- Simplified consistency models
- Eventual consistency

**Types of NoSQL:**

Key-Value Stores: Cassandra, Riak

Document Stores: MongoDB, CouchDB

Tabular Stores: HBase, BigTable

**The Trend: Recognizing SQL's power for non-relational data**

# Teaching SQL on NoSQL: Impala & SlamData

## Impala for HBase

- Fast SQL engine on HBase/HDFS
- Better for interactive queries than Hive
- Shares metadata with Hive

**Demo:** Querying U.S. occupations data

## SlamData for MongoDB

- SQL on document-based MongoDB
- Handles JSON/BSON documents
- Reduces need for joins

**Demo:** IRS tax data by ZIP code

**Note:** Code and demo materials available in the paper

# NewSQL Systems

# The Best of Both Worlds: NewSQL

## NewSQL: Combining the Best Features

- **Promise:** NoSQL scalability + RDBMS reliability
- **From Traditional RDBMS:**
  - ACID properties (Atomicity, Consistency, Isolation, Durability)
  - Full SQL support
  - Relational data model
- **From NoSQL Systems:**
  - High scalability and availability
  - Distributed, fault-tolerant architecture
  - Horizontal scaling capabilities
- **Result:** Systems that handle massive workloads while maintaining data integrity

# NewSQL Categories

**New Architectures**

- VoltDB
- NuoDB

**Optimized SQL Engines**

- MySQL Cluster
- Infobright

**Transparent Sharding**

- dbShards
- ScaleBase

**Teaching Benefit:** Familiar examples (employee/university DBs) at massive scale!

# Demo: Learning SQL with VoltDB

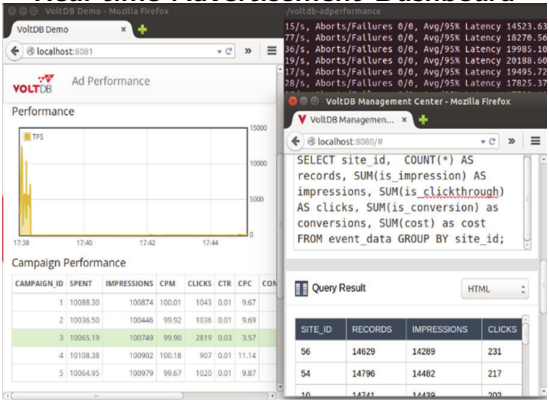
**VoltDB Features:**

- In-memory, ACID-compliant
- Shared-nothing architecture
- Real-time analytics on fast data

**Teaching Tools:**

- VoltDB VM for simulation
- Application Gallery with real use cases

**Real-time Advertisement Dashboard**




Processes data *as it flows in*

# Conclusion

# Key Takeaways

- SQL is no longer just for traditional databases
- It's a **universal language** for the Big Data era
- Must teach across **MapReduce**, **NoSQL**, and **NewSQL**
- Practical approach using VMs and synthetic data
- Prepares students for real-world distributed systems

**Educational Impact**

Empowers students to extract insights from data on **any platform**

# Integration into Curriculum

**Three Approaches:**

1. **Replacement:** Use NewSQL instead of traditional RDBMS
2. **Complement:** Teach fundamentals with traditional, then extend to Big Data
3. **Dedicated Course:** Big Data course covering all three paradigms

# Thank You!

Questions?

**Paper:** SQL: From Traditional Databases to Big Data
Paper Link